

Working with `localStorage` in JavaScript

The `localStorage` API allows you to store key-value data in the browser, with no expiration date. This data persists across page reloads and browser sessions, unless manually cleared.

Key Features

- Stores data as **strings**
- Synchronous API (blocking)
- Maximum storage capacity: around 5MB (varies by browser)
- Data is domain-specific

Basic Syntax

```
// Set item
localStorage.setItem('key', 'value');

// Get item
const value = localStorage.getItem('key');

// Remove item
localStorage.removeItem('key');

// Clear all items
localStorage.clear();
```

Example: Storing and Retrieving Data

```
localStorage.setItem('username', 'haris');

const user = localStorage.getItem('username');
console.log(user); // Output: haris
```

Storing Objects

Since `localStorage` only stores strings, you must convert objects using `JSON.stringify()` and retrieve them using `JSON.parse()` :

```
const user = {
  name: 'Harry',
  age: 25
};

// Store the object
localStorage.setItem('user', JSON.stringify(user));

// Retrieve the object
const storedUser = JSON.parse(localStorage.getItem('user'));
console.log(storedUser.name); // Output: Harry
```

Checking if Key Exists

```
if (localStorage.getItem('theme')) {
  console.log('Theme is set');
}
```

Use Cases

- Saving user preferences (e.g., theme, language)

- Caching small amounts of data
- Keeping users logged in (store tokens)
- Temporarily saving form data

Limitations

- Synchronous (can block main thread)
- Storage limit (~5MB)
- No automatic expiration (unlike `sessionStorage` or cookies)
- Not secure for sensitive data (easily inspectable via DevTools)

Summary

- `localStorage` provides simple, persistent browser storage.
- Use `setItem`, `getItem`, `removeItem`, and `clear` methods.
- Always stringify objects before storing.
- Avoid storing sensitive data.